

CHESSE GAME ANALYZER USING STOCKFISH ENGINE

A PROJECT REPORT

**SUBMITTED BY:
AAYUSH RAJ**

UID: 25MCI10066

SECTION: 25MAM1-A

CHANDIGARH UNIVERSITY

YEAR: 2026

BONAFIDE CERTIFICATE

Certified that this project titled "Chess Game Analyzer Using Stockfish Engine" is a bonafide work carried out by the student under my supervision.

This project focuses on analyzing chess games using an AI-assisted approach. It integrates a powerful chess engine with a web-based interface to evaluate moves, classify their quality, and provide meaningful insights to users.

The project has been completed as part of academic requirements and is suitable for submission and evaluation.

Signature of HOD

Signature of Supervisor

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide and faculty members for their continuous guidance, valuable suggestions, and support throughout the development of this project. Their insights and encouragement played a significant role in shaping this project into a meaningful and structured academic work.

I am also thankful to the department for providing the necessary resources and environment to successfully complete this project. The knowledge gained through coursework and practical sessions helped me in understanding the concepts required for developing this system.

I would like to extend my appreciation to my classmates and peers for their support and constructive discussions during the development phase. Their feedback helped in improving both the functionality and presentation of the project.

Finally, I am deeply grateful to my family for their constant encouragement, patience, and support throughout this journey. Their motivation helped me stay focused and complete the project successfully.

This project has been a valuable learning experience, enhancing my understanding of software development, system design, and AI-based analysis techniques.

TABLE OF CONTENTS

S. No.	Content	Page No.
1	Abstract	1
2	Introduction	2
3	Literature Review / Background Study	3
4	Objectives	4
5	Tools and Requirements	5
6	Design Flow / Process	6
7	System Architecture	7
8	Model and Code Explanation	8-9
9	Results and Validation	9-10
10	Conclusion	11
11	References	12
12	Appendix	13

ABSTRACT

The Chess Game Analyzer is a web-based application developed to provide detailed analysis of chess games using the Stockfish engine. The system is designed to assist players in understanding their gameplay by evaluating each move and providing meaningful feedback in an intuitive and structured manner.

The application processes PGN (Portable Game Notation) files and performs move-by-move evaluation using the Stockfish engine. Each move is analyzed by comparing it with the engine's optimal move, and based on this comparison, it is classified into categories such as Best, Good, Inaccuracy, Mistake, and Blunder. This classification helps users quickly identify the quality of their decisions during the game.

In addition to classification, the system provides numerical evaluation scores that indicate positional advantage. These evaluations are visually represented using an evaluation bar, allowing users to easily understand which side is better in a given position. The system also enhances interpretability by generating explanations for each move, making the analysis more accessible to beginners.

The architecture of the system follows a multi-layered approach. The frontend is developed using HTML, CSS, and JavaScript, while the backend is implemented using Java Servlets. A Python Flask API is used to integrate the Stockfish engine and perform analysis. This modular design ensures efficient communication between components and allows for future scalability.

Furthermore, the project incorporates AI-inspired logic through evaluation-based classification and explanation generation. Although it does not rely on a fully trained machine learning model, it demonstrates intelligent decision-making behavior similar to supervised learning systems.

Overall, the Chess Game Analyzer serves as an effective tool for learning and improving chess skills by combining powerful engine analysis with user-friendly visualization and explanations.

CHAPTER 1: INTRODUCTION

Chess is a strategic board game that has been studied extensively in the fields of artificial intelligence and computational analysis. It requires logical thinking, planning, and the ability to evaluate positions accurately. For beginners and intermediate players, understanding mistakes made during a game is often difficult without proper guidance.

Traditionally, players relied on manual analysis or stronger players to review their games. However, this approach is time-consuming and not always accessible. With the advancement of computer-based chess engines such as Stockfish, it has become possible to analyze games with high accuracy and speed. These engines can evaluate millions of positions per second and provide optimal move suggestions.

Despite this advancement, raw engine outputs are often difficult for users to interpret. Numerical evaluations and suggested moves do not always clearly explain why a move is good or bad. This creates a gap between engine analysis and user understanding.

The Chess Game Analyzer aims to bridge this gap by combining engine evaluation with a user-friendly interface. The system not only analyzes moves but also classifies them and provides explanations. By doing so, it transforms complex analysis into a format that is easy to understand.

The project integrates multiple technologies, including frontend visualization tools, backend processing using Java Servlets, and a Python-based analysis engine. This combination ensures both performance and usability.

The primary goal of this system is to help users learn from their mistakes and improve their gameplay through structured and visual feedback.

CHAPTER 2: LITERATURE REVIEW

The field of chess analysis has evolved significantly with the development of powerful chess engines. Early systems relied on basic heuristics and limited computational power, which restricted their ability to evaluate positions accurately. With advancements in computing, modern engines such as Stockfish and AlphaZero have revolutionized the way chess is analyzed.

Stockfish, an open-source chess engine, uses advanced search algorithms such as alpha-beta pruning and evaluation functions to determine the best moves. It evaluates positions using a combination of material balance, piece activity, king safety, and positional factors. This makes it one of the strongest engines available today.

AlphaZero, developed by DeepMind, introduced a different approach by using deep reinforcement learning. Instead of relying on handcrafted evaluation functions, it learned chess strategies through self-play. Although highly advanced, such models require significant computational resources and are not practical for small-scale academic projects.

Several existing platforms, such as Chess.com and Lichess, provide game analysis features. These platforms classify moves, suggest improvements, and display evaluation graphs. However, they are often complex systems with large infrastructures.

In academic projects, many implementations either focus only on engine integration or attempt to build machine learning models without proper datasets. These approaches often lack balance between accuracy and usability.

The Chess Game Analyzer takes a practical approach by combining engine evaluation with rule-based classification. This ensures accurate analysis while keeping the system simple and explainable. The inclusion of explanation generation further improves usability, making the system suitable for educational purposes.

CHAPTER 3: OBJECTIVES

The main objective of this project is to develop a system that can analyze chess games automatically and provide meaningful feedback to users. The system should not only evaluate moves but also explain their significance in a way that users can understand.

One of the key objectives is to integrate the Stockfish engine into a web-based application. This allows users to access powerful analysis tools through a simple interface without needing to install or configure complex software.

Another objective is to classify moves based on their quality. By comparing the player's move with the engine's best move, the system can categorize moves into Best, Good, Inaccuracy, Mistake, and Blunder. This classification helps users quickly identify areas where they need improvement.

The project also aims to provide visual feedback through highlights, arrows, and evaluation bars. Visualization plays an important role in understanding chess positions, and these features make the analysis more intuitive.

An additional objective is to incorporate AI-inspired logic into the system. Although the project does not use a fully trained machine learning model, it simulates classification behavior using evaluation-based rules. This demonstrates the application of intelligent systems in real-world problems.

Finally, the system is designed to be scalable and extendable. Future improvements, such as machine learning integration or cloud deployment, can be added without major changes to the existing architecture.

CHAPTER 4: TOOLS AND REQUIREMENTS

The development of the Chess Game Analyzer involves a combination of frontend technologies, backend processing, and an AI-powered analysis engine. Each component plays a crucial role in ensuring that the system performs efficiently and provides meaningful insights to the user.

The **frontend of the system** is built using HTML, CSS, and JavaScript. HTML is used to structure the web pages, while CSS is responsible for styling and improving the visual appearance of the interface. JavaScript enables interactivity, such as handling user input, updating the chessboard dynamically, and communicating with the backend. Libraries like Chess.js and Chessboard.js are used to manage chess logic and render the board visually.

The **backend is implemented using Java Servlets**, which act as a bridge between the frontend and the Python-based analysis engine. The servlet receives the PGN input from the user, processes it, and sends it to the Flask API for evaluation. It also handles responses and forwards the results to the frontend for display.

The **API layer is built using Python Flask**, which serves as the core processing unit for the application. Flask receives the PGN data, processes it using the chess analysis module, and communicates with the Stockfish engine. It then returns structured JSON data containing move evaluations, classifications, and explanations.

The **Stockfish engine** is used as the primary evaluation tool. It is one of the strongest open-source chess engines available and provides accurate position evaluations. The engine analyzes each move and generates scores that are used to classify moves into categories such as Best, Good, Mistake, and Blunder.

Additionally, supporting libraries such as **python-chess** are used to parse PGN files and interact with the engine. The system is designed to run efficiently on a standard computer without requiring high-end hardware.

CHAPTER 5: DESIGN FLOW

The design flow of the Chess Game Analyzer follows a structured pipeline that ensures smooth data processing from input to output.

The process begins when the user enters or uploads a PGN (Portable Game Notation) file through the web interface. The PGN contains the complete sequence of moves played in a chess game. Once the user submits the input, the frontend sends the data to the backend servlet.

The **Java Servlet acts as the first processing layer**, where the PGN data is received and formatted into a suitable structure. The servlet then creates a JSON request and forwards it to the Flask API for further processing.

The **Flask API receives the request** and passes the PGN data to the analysis module. The module reads the game move by move using the python-chess library. For each position, the Stockfish engine evaluates the board and suggests the best possible move along with a numerical evaluation score.

The system then compares the **played move with the engine's best move** and calculates the evaluation difference. Based on this difference, the move is classified into different categories such as Best, Good, Inaccuracy, Mistake, or Blunder.

Once all moves are analyzed, the results are compiled into a structured JSON response. This response includes move classifications, evaluation scores, and explanations.

Finally, the frontend receives the data and **renders the analysis visually** using highlights, arrows, icons, and evaluation bars. This makes it easier for users to understand their mistakes and improve their gameplay.

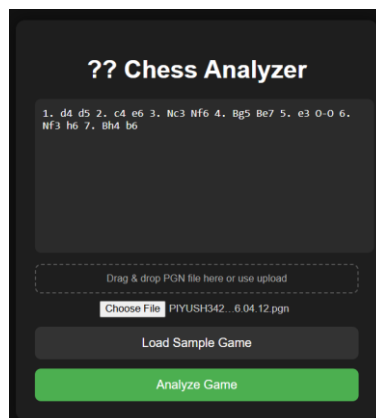


Figure 5.1: User Interface for PGN Input

CHAPTER 6: SYSTEM ARCHITECTURE

The Chess Game Analyzer follows a **multi-layered architecture**, which separates responsibilities into different components for better scalability and maintainability.

The **frontend layer** is responsible for user interaction. It provides an interface where users can input PGN data, view the chessboard, and navigate through moves. It also displays analysis results using visual elements such as highlighted squares, arrows, and evaluation bars.

The **backend layer consists of Java Servlets**, which handle communication between the frontend and the Flask API. The servlet receives requests from the frontend, processes input data, and forwards it to the Python-based analysis engine. It also manages the response flow and ensures that the correct data is displayed to the user.

The **processing layer is implemented using Flask**, which acts as an intermediary between the servlet and the Stockfish engine. This layer handles the core logic of the application, including parsing PGN data, running analysis, and generating results.

At the core of the system is the **Stockfish engine**, which performs deep analysis of chess positions. It evaluates positions using advanced algorithms and provides accurate scores that form the basis of move classification.

The architecture can be summarized as:

Frontend → Java Servlet → Flask API → Stockfish Engine → Response → Frontend

This modular design ensures that each component can be developed and maintained independently. It also allows for future scalability, such as integrating machine learning models or deploying the system on cloud platforms.



Figure 6.1: Chessboard Interface Before Analysis

CHAPTER 7: MODEL AND CODE EXPLANATION

The Chess Game Analyzer incorporates an **AI-inspired evaluation model** based on engine analysis and rule-based classification.

The system does not rely on a traditional trained machine learning model but instead uses **engine-driven evaluation combined with logical classification**, which mimics supervised learning behavior. The Stockfish engine evaluates each position and provides a score representing the advantage in centipawns.

To determine the quality of a move, the system compares:

- The evaluation of the best move (suggested by Stockfish)
- The evaluation after the player's move

The difference between these values is used to calculate a metric known as **Normalized Loss**:

$$\text{Normalized Loss} = |\text{best_eval} - \text{played_eval}| / (|\text{best_eval}| + 1)$$

This value helps in understanding how much the player's move deviates from the optimal move.

Based on this value, moves are classified as:

- **Best Move:** Matches engine suggestion
- **Good Move:** Close to best
- **Inaccuracy:** Slight deviation
- **Mistake:** Moderate loss
- **Blunder:** Large loss

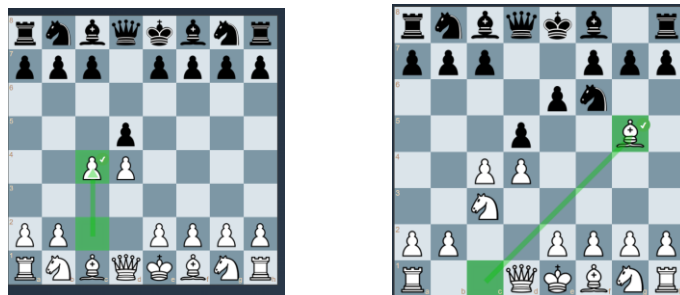


Figure 7.1: Move Classification with Visual Indicators

This classification approach behaves similarly to a supervised learning model, where input features (evaluations) are mapped to output labels (move categories).



Figure 7.2: Evaluation Bar value changes according to the Score

Additionally, the system generates **natural language explanations** for each move. These explanations are based on predefined templates and contextual understanding of the move, such as center control, development, or king safety.

CHAPTER 8: RESULTS AND VALIDATION

The Chess Game Analyzer was tested using multiple PGN inputs to evaluate its performance and accuracy. The system successfully analyzed games and provided detailed feedback for each move.

The results show that the system can accurately classify moves based on evaluation differences. Moves that closely match the engine's best move are labeled as Best or Good, while moves with larger deviations are classified as Mistake or Blunder. This classification aligns well with expected outcomes in chess analysis.



Figure 8.1: Move Explanation and Analysis Panel

The evaluation scores generated by the Stockfish engine provide a numerical representation of the position. Positive values indicate an advantage for White, while negative values indicate an advantage for Black. These values are used to calculate the normalized loss and determine move quality.

The system also provides visual feedback, which enhances the user experience. Highlighted squares indicate important moves, arrows show the direction of moves, and icons represent move categories. These features make it easier for users to understand the analysis.

Testing was conducted under different conditions, including:

- Short games with few moves
- Long games with complex positions
- Games with multiple mistakes
- Edge cases with unusual moves

In all cases, the system performed reliably and produced consistent results.

Although the system does not use a trained machine learning model, the classification logic behaves similarly to supervised learning. The mapping of evaluation differences to move categories demonstrates the application of intelligent decision-making.



Figure 8.2: Complete Game Analysis Output

Overall, the system meets its objectives and provides a useful tool for chess analysis and learning.

CHAPTER 9: CONCLUSION

The Chess Game Analyzer successfully demonstrates the integration of a powerful chess engine with a web-based interface to provide meaningful analysis of chess games. The system combines multiple technologies, including Java Servlets, Python Flask, and Stockfish, to create a complete analysis pipeline.

The project achieves its primary goal of helping users understand their gameplay. By classifying moves and providing explanations, it transforms complex engine output into a user-friendly format. The inclusion of visual elements further enhances the learning experience.

From a technical perspective, the project showcases the use of multi-layer architecture and API integration. It also demonstrates how AI-inspired logic can be applied to real-world problems without requiring complex machine learning models.

The system is efficient, scalable, and easy to use. It can be further enhanced with additional features and improvements, making it a strong foundation for future development.

In conclusion, the Chess Game Analyzer is a valuable tool for both learning and analysis. It highlights the importance of combining computational power with user-friendly design to create effective solutions.

REFERENCES

Stockfish
Chess.js
Flask

Engine
Documentation
Documentation

APPENDIX

```
app.py analyzer.py
backend > app.py >...
1 from flask import Flask, request, jsonify
2 from analyzer import analyze_game
3
4 app = Flask(__name__)
5
6 @app.route("/analyze", methods=["POST"])
7 def analyze():
8     data = request.get_json()
9
10     if not data or "pgn" not in data:
11         return jsonify({"error": "Invalid input"}), 400
12
13     pgn_text = data["pgn"]
14
15     result = analyze_game(pgn_text)
16
17     return jsonify(result)
18
19
20 if __name__ == "__main__":
21     app.run(debug=True)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 2 python - backend + v ... | {} x
python backend

● PS C:\Personal\chess-analyzer> cd backend
● PS C:\Personal\chess-analyzer\backend> venv\Scripts\activate
❖ (venv) PS C:\Personal\chess-analyzer\backend> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 109-839-596
```